ICS 202 – Data Structures and Algorithms

Topic 06: Analysis of Recursive Algorithms

Question bank: 40 multiple choice + 10 programming

Questions from ICS 202 old exams, quizzes and activities (AI_Work). **[new]** marks new questions written in the same style.

Part A: Multiple choice

Q1. [new]

In a recurrence relation $T(n)$, the *base case* is

(A) the portion of the definition that contains T

(B) the portion of the definition that does not contain T

(C) the closed form of the recurrence relation

(D) the largest value of n for which the relation is defined

Answer: (B). The base case is the part that does not contain T; the recursive case is the part that contains T.

My answer

Q2. [new]

Which of the following is **not** one of the four methods for solving recurrence relations that represent the running time of recursive methods?

(A) Iteration method (unrolling and summing)

(B) Recursion tree method

(C) Binary partition method

(D) Master method

Answer: (C). The four methods are iteration, substitution, recursion tree and master method.

My answer

Q3. [new]

What is the recurrence relation describing the number of **comparisons** carried out by the following method?

```
public void f(int n) {
   if (n > 0) {
      System.out.println(n);
      f(n - 1);
```

```
    }
}
```

(A) $T(0) = 0, \quad T(n) = T(n-1) + 1$ for $n > 0$

(B) $T(0) = 1, \quad T(n) = T(n-1)$ for $n > 0$

(C) $T(0) = 1, \quad T(n) = T(n-1) + 1$ for $n > 0$

(D) $T(0) = 1, \quad T(n) = 2T(n-1) + 1$ for $n > 0$

Answer: (C). The base case still performs the comparison $n > 0$, so $T(0) = 1$; each other call does one comparison plus $T(n-1)$.

My answer

Q4. [new]

For the same method `f` of the previous question, what is the recurrence relation describing the number of `System.out.println` statements executed?

(A) $T(0) = 0, \quad T(n) = T(n-1) + 1$ for $n > 0$

(B) $T(0) = 1, \quad T(n) = T(n-1) + 1$ for $n > 0$

(C) $T(0) = 0, \quad T(n) = T(n-1)$ for $n > 0$

(D) $T(0) = 1, \quad T(n) = T(n-1) + 2$ for $n > 0$

Answer: (A). Nothing is printed when $n = 0$; otherwise one print plus the prints of $f(n-1)$.

My answer

Q5. [new]

What is the recurrence relation describing the number of **additions** carried out by the following method (assume n is a power of 2)?

```
public int g(int n) {
   if (n == 1)
      return 2;
   else
      return 3 * g(n / 2) + g(n / 2) + 5;
}
```

(A) $T(1) = 0, \quad T(n) = T(n/2) + 2$ for $n > 1$

(B) $T(1) = 0, \quad T(n) = 2T(n/2) + 2$ for $n > 1$

(C) $T(1) = 1, \quad T(n) = 2T(n/2) + 3$ for $n > 1$

(D) $T(1) = 0, \quad T(n) = 2T(n/2) + 1$ for $n > 1$

Answer: (B). Two recursive calls on $n/2$ and two + operations per call; no additions in the base case.

My answer

Q6.

Unit 06 Activity 01 (T252)

What is the recurrence equation that represents the number of arithmetic operations (`*` and `-`) carried out by the following method?

```
static int power(int x, int n) {
   if (n == 0)   // Base Condition
      return 1;
   return x * power(x, n - 1);
}
```

(A) $T(0) = 0, \quad T(n) = T(n-1) + 2$ for $n > 0$

(B) $T(0) = 1, \quad T(n) = T(n-1) + 1$ for $n > 0$

(C) $T(0) = 0, \quad T(n) = 2T(n-1) + 1$ for $n > 0$

(D) $T(0) = 0, \quad T(n) = T(n/2) + 2$ for $n > 0$

Answer: (A). Each non-base call does one multiplication and one subtraction ($n - 1$); this gives $T(n) = 2n = O(n)$.

My answer

Q7.

Midterm 202

Which recurrence relation represents the number of additions $T(n)$ carried out by the following method (assume n is even)?

```
public static int myMethod(int n){
   if(n == 0)
      return 0;
   else{
      System.out.println(n);
      return myMethod(n - 2) + n;
   }
}
```

(A) $T(0) = 0, \quad T(n) = T(n-1) + 1$ for $n > 0$

(B) $T(0) = 0, \quad T(n) = T(n-2) + 2$ for $n > 0$

(C) $T(0) = 0, \quad T(n) = T(n-2) + n$ for $n > 0$

(D) $T(0) = 0, \quad T(n) = T(n-2) + 1$ for $n > 0$

Answer: (D). One addition (`+ n`) per call plus the additions of `myMethod(n - 2)`; the subtraction $n - 2$ is not an addition.

My answer

Q8. Quiz 3 V1 (T251)

What is the correct recurrence relation for the `recur` method considering the number of multiplications carried out by the method?

```
int recur(int n) {
   if (n <= 1)
      return 1;
   else
      return recur(n/2) * recur(n/2);
}
```

(A) $T(n) = T(n-1) + 1$ with $T(1) = 0$

(B) $T(n) = 2T(n/2) + 1$ with $T(1) = 0$

(C) $T(n) = T(n/2) + 1$ with $T(1) = 0$

(D) $T(n) = T(n-1) + T(n-2) + 1$ with $T(1) = 0$

Answer: (B). Two calls on $n/2$ and one multiplication per non-base call.

My answer

Q9. Recursion compilation (T181)

Which recurrence relation represents the time complexity $T(n)$ of the following method, counting the additions? Assume n is a power of 2.

```
int mystery(int n) {
   if (n == 1)
      return n;
   return mystery(n/2) + mystery(n/2);
}
```

(A) $T(1) = 0, \quad T(n) = T(n/2) + 1$ for $n > 1$

(B) $T(1) = 0, \quad T(n) = 2T(n/2) + 2$ for $n > 1$

(C) $T(1) = 0, \quad T(n) = 2T(n/2) + 1$ for $n > 1$

(D) $T(1) = 1, \quad T(n) = 2T(n-1) + 1$ for $n > 1$

Answer: (C). One addition per call and two recursive calls on $n/2$.

My answer

Q10. Recursion compilation (T211)

Which recurrence relation represents the running time $T(n)$ of the function below in terms of the number of times statement `S1` is executed? (Assume n is a power of 2.)

```
void test(int n){
   if(n > 1){
      for(int k = 0; k < n; k++){
         System.out.printf("%d ", k); //S1
```

```
        }
        test(n/2);
        test(n/2);
    }
}
```

(A) $T(1) = 0, \quad T(n) = T(n/2) + n$ for $n > 1$

(B) $T(1) = 0, \quad T(n) = 2T(n/2) + n$ for $n > 1$

(C) $T(1) = 0, \quad T(n) = 2T(n/2) + 1$ for $n > 1$

(D) $T(1) = 0, \quad T(n) = 2T(n-1) + n$ for $n > 1$

Answer: (B). The loop executes `S1` n times, then two calls on $n/2$; nothing is printed when $n = 1$.

My answer

Q11.

Unit 06 Activity 01 (T252)

For the method `test` of the previous question, what is the big-O complexity of the number of times `S1` is executed?

(A) $O(n)$

(B) $O(\log n)$

(C) $O(n^2)$

(D) $O(n \log n)$

Answer: (D). $T(n) = 2^k T(n/2^k) + kn$; stopping at $k = \log n$ gives $T(n) = n \log n$.

My answer

Q12.

Unit 06 Activity 01 (T252)

What is the recurrence equation that represents the number of times the print statement gets executed?

```
public static void f(String s, int n){
    if (n >= 1) {
        System.out.print(s.charAt(n));
        f(s, n - 1);
        System.out.print(s.charAt(n-1));
        f(s, n - 1);
    }
}
```

(A) $T(0) = 0, \quad T(n) = T(n-1) + 2$ for $n \geq 1$

(B) $T(0) = 0, \quad T(n) = 2T(n-1) + 1$ for $n \geq 1$

(C) $T(0) = 0, \quad T(n) = 2T(n-1) + 2$ for $n \geq 1$

(D) $T(0) = 0, \quad T(n) = 2T(n/2) + 2$ for $n \geq 1$

Answer: (C). Two prints and two calls on $n-1$ per call; this solves to $2^{n+1}-2 = O(2^n)$.

My answer

Q13.

Major 1 (T112)

Consider the following recursive method:

```
public static void f(String s, int index){
   if (index > 0) {
      f(s, index - 1);
      System.out.print(s.charAt(index));
      f(s, index - 2);
      System.out.print(s.charAt(index - 1));
   }
}
```

Which recurrence equation describes the number of times the print statement is executed?

(A) $T(n) = 0$ for $n \leq 0$, $\quad T(n) = T(n-1) + T(n-2) + 1$ for $n > 0$

(B) $T(n) = 0$ for $n \leq 0$, $\quad T(n) = T(n-1) + T(n-2) + 2$ for $n > 0$

(C) $T(n) = 0$ for $n \leq 0$, $\quad T(n) = 2T(n-1) + 2$ for $n > 0$

(D) $T(n) = 0$ for $n \leq 0$, $\quad T(n) = T(n-1) + 2$ for $n > 0$

Answer: (B). Calls on $n-1$ and $n-2$ plus two prints per call (e.g., `f("KFUPM", 3)` prints 8 characters).

My answer

Q14.

Unit 06 Activity 01 (T252)

What is the recurrence equation that represents the number of arithmetic operations (+ and −) carried out by the following method?

```
int fib(int N) {
   if ((N == 1) || (N == 2)) {
      return 1;
   } else {
      return (fib(N-1) + fib(N-2));
   }
}
```

(A) $T(1) = T(2) = 1$, $\quad T(N) = T(N-1) + T(N-2) + 1$ for $N > 2$

(B) $T(1) = T(2) = 0$, $\quad T(N) = 2T(N-1) + 3$ for $N > 2$

(C) $T(1) = T(2) = 1$, $\quad T(N) = T(N-1) + T(N-2) + 3$ for $N > 2$

(D) $T(1) = T(2) = 0$, $\quad T(N) = T(N-1) + T(N-2) + 3$ for $N > 2$

Answer: (D). No arithmetic in the base case; otherwise $N-1$, $N-2$ and one +, plus the two calls.

My answer

Q15.

Recursion compilation (T172)

Consider the following recursive algorithm `weird`. We want its time complexity in terms of the number of element additions carried out in lines 6 and 11.

```
1.  static int weird(int[] A, int n) {
2.     if (n <= 0)
3.        return 1;
4.     else {
5.        if (n is even) {
6.           return A[n] + weird(A, n/2);
7.        } else
8.        {
9.           int k = 0;
10.          for i = 1 to n
11.             k = k + A[i];
12.          return k * weird(A, n-2);
13.       }
14.    }
```

Which recurrence equation describes the **best case** time complexity of the algorithm?

(A) $T(n) = 0$ for $n \leq 0$, $\quad T(n) = T(n/2) + 1$ for $n > 0$

(B) $T(n) = 0$ for $n \leq 0$, $\quad T(n) = T(n-2) + n$ for $n > 0$

(C) $T(n) = 0$ for $n \leq 0$, $\quad T(n) = T(n/2) + n$ for $n > 0$

(D) $T(n) = 0$ for $n \leq 0$, $\quad T(n) = T(n-2) + 1$ for $n > 0$

Answer: (A). The best case is the even branch: one addition (line 6) and a call on $n/2$.

My answer

Q16.

Recursion compilation (T172)

For the algorithm `weird` of the previous question, which recurrence equation describes the **worst case** time complexity?

(A) $T(n) = 0$ for $n \leq 0$, $\quad T(n) = T(n/2) + n$ for $n > 0$

(B) $T(n) = 0$ for $n \leq 0$, $\quad T(n) = T(n-2) + 1$ for $n > 0$

(C) $T(n) = 0$ for $n \leq 0$, $\quad T(n) = T(n-2) + n$ for $n > 0$

(D) $T(n) = 0$ for $n \leq 0$, $\quad T(n) = 2T(n-2) + n$ for $n > 0$

Answer: (C). The worst case is the odd branch: the loop does n additions (line 11), then one call on $n-2$.

My answer

Q17.

Recursion compilation (T152)

Consider the following summarized version of algorithm `mergeSort`:

```
static void mergeSort(int[] A) {
   if (A.length > 1) {
      int q = A.length/2;
      int[] leftArray = Arrays.copyOfRange(A, 0, q);
      int[] rightArray = Arrays.copyOfRange(A, q, A.length);
      mergeSort(leftArray);
      mergeSort(rightArray);
      A = merge(leftArray, rightArray);
   }
}
```

Assume the `merge` function carries out $n-1$ element comparisons when `mergeSort` is called to sort n elements. Which recurrence describes the worst-case number of element comparisons?

(A) $T(1) = 0, \quad T(n) = T(n/2) + (n-1)$

(B) $T(1) = 0, \quad T(n) = 2T(n/2) + (n-1)$

(C) $T(1) = 0, \quad T(n) = 2T(n/2) + 1$

(D) $T(1) = 1, \quad T(n) = 2T(n-1) + n$

Answer: (B). Two recursive calls on halves plus $n-1$ comparisons in `merge`; no comparisons for one element.

My answer

Q18.

[new]

What is the recurrence relation describing the number of times the + operator is executed by the following method?

```
static int sum(int[] a, int n) {
   if (n == 0)
      return 0;
   return sum(a, n - 1) + a[n - 1];
}
```

(A) $T(0) = 0, \quad T(n) = T(n-1) + 1$ for $n > 0$; so $T(n) = n$

(B) $T(0) = 1, \quad T(n) = T(n-1) + 1$ for $n > 0$; so $T(n) = n + 1$

(C) $T(0) = 0, \quad T(n) = T(n-1) + n$ for $n > 0$; so $T(n) = n(n+1)/2$

(D) $T(0) = 0, \quad T(n) = 2T(n-1) + 1$ for $n > 0$; so $T(n) = 2^n - 1$

Answer: (A). One + per non-base call and one recursive call on $n-1$; $T(n) = n = O(n)$.

My answer

Q19.

[new]

Assume n is a power of 2. What is the number of times the print statement is executed by `h(n)`, and its big-O complexity?

```
static void h(int n) {
   if (n > 1) {
      for (int i = 0; i < n; i++)
         System.out.println(i);
      h(n / 2);
   }
}
```

(A) $T(n) = n \log n$, which is $O(n \log n)$

(B) $T(n) = 2n - 2$, which is $O(n)$

(C) $T(n) = \log n$, which is $O(\log n)$

(D) $T(n) = n^2 - 1$, which is $O(n^2)$

Answer: (B). $T(1) = 0$, $T(n) = T(n/2) + n = n + n/2 + \cdots + 2 = 2n - 2 = O(n)$.

My answer

Q20. [new]

What is the closed form of the number of print statements executed by `p(n)`?

```
static void p(int n) {
   if (n > 0) {
      System.out.println(n);
      p(n - 1);
      p(n - 1);
      p(n - 1);
   }
}
```

(A) $3n$

(B) $3^n - 1$

(C) $\dfrac{3^n - 1}{2}$

(D) $2^n - 1$

Answer: (C). $T(0) = 0$, $T(n) = 3T(n-1) + 1 = \sum_{i=0}^{n-1} 3^i = (3^n - 1)/2$, which is $O(3^n)$.

My answer

Q21. [new]

When solving $T(n) = T(n/2) + 1$ with $T(0) = 0$ by iteration, we get $T(n) = T(n/2^k) + k$. Why do we stop when $n/2^k = 1$ instead of when $n/2^k = 0$?

(A) Because $T(1)$ is always undefined

(B) Because $n/2^k = 0$ cannot be solved for k in terms of n, so we stop at one element and compute $T(1) = T(0) + 1 = 1$

(C) Because stopping at $n/2^k = 1$ gives a smaller big-O

(D) Because the iteration method only works when the base case is $T(1)$

Answer: (B). $n/2^k$ never equals 0 for finite k; stopping at $n = 2^k$ gives $k = \log_2 n$ and $T(n) = 1 + \log_2 n$.

My answer

Q22. [new]

When solving $T(n) = 2T(n-1) + 1$ by the iteration method, what is $T(n)$ after k expansions?

(A) $2^k T(n-k) + k$

(B) $2^k T(n-k) + 2^k - 1$

(C) $kT(n-k) + 2^k$

(D) $2T(n-k) + 2^k - 1$

Answer: (B). $T(n) = 2^k T(n-k) + 2^{k-1} + \cdots + 2 + 1 = 2^k T(n-k) + (2^k - 1)$.

My answer

Q23. Midterm 222

What is the closed form solution of the recurrence relation $T(n) = T(n-1) + 7$, $T(1) = 4$?

(A) $T(n) = 7n + 3$, $n \geq 1$

(B) $T(n) = 7n + 4$, $n \geq 0$

(C) $T(n) = 3n + 1$, $n \geq 1$

(D) $T(n) = 7n - 3$, $n \geq 1$

Answer: (D). $T(n) = T(n-k) + 7k$; stop at $k = n - 1$: $T(n) = 4 + 7(n-1) = 7n - 3$.

My answer

Q24. Midterm 202

The running time $T(n)$ of an algorithm is represented by the recurrence relation $T(0) = a$, $T(n) = T(n-1) + \frac{n}{2} + b$ for $n > 0$, where a and b are constants. Solving by iteration gives

(A) $T(n) = a + \frac{n(n+1)}{4} + nb$, which is $O(n^2)$

(B) $T(n) = a + \frac{n(n+1)}{2} + nb$, which is $O(n^2)$

(C) $T(n) = a + \frac{n}{2} + nb$, which is $O(n)$

(D) $T(n) = a + \frac{n^2}{4} + b$, which is $O(n^2)$

Answer: (A). $T(n) = T(n-k) + \frac{1}{2}\sum_{i=n-k+1}^{n} i + kb$; with $k = n$: $a + \frac{n(n+1)}{4} + nb = O(n^2)$.

My answer

Q25. [new]

What is the closed form of $T(n) = T(n-1) + n$ for $n > 0$, with $T(0) = 0$?

(A) $T(n) = n^2$

(B) $T(n) = 2^n - 1$

(C) $T(n) = \dfrac{n(n+1)}{2}$

(D) $T(n) = n \log n$

Answer: (C). $T(n) = T(0) + 1 + 2 + \cdots + n = n(n+1)/2 = O(n^2)$.

My answer

Q26. Midterm 222

What is the closed form solution of the recurrence relation $T(n) = 2T(n/2) + n$, $T(1) = 1$ (n a power of 2)?

(A) $T(n) = n^2 \log n$, $n \geq 1$

(B) $T(n) = n \log n$, $n \geq 1$

(C) $T(n) = n + n \log n$, $n \geq 1$

(D) $T(n) = n^n \log n$, $n \geq 1$

Answer: (C). $T(n) = 2^k T(n/2^k) + kn$; at $k = \log n$: $nT(1) + n \log n = n + n \log n = \Theta(n \log n)$.

My answer

Q27. Major 1 (T112)

Solve the recurrence relation $T(1) = 1,\ T(n) = 2T(n/2) + 1$ for $n > 1$ (n a power of 2). What are its closed form and big-O complexity?

(A) $T(n) = 2n - 1$, which is $O(n)$

(B) $T(n) = n - 1$, which is $O(n)$

(C) $T(n) = n \log n + 1$, which is $O(n \log n)$

(D) $T(n) = \log n + 1$, which is $O(\log n)$

Answer: (A). $T(n) = 2^k T(n/2^k) + (2^k - 1)$; at $2^k = n$: $n + n - 1 = 2n - 1$.

My answer

Q28. Recursion compilation (T181)

Solve the recurrence of method `mystery` (counting additions): $T(1) = 0,\ T(n) = 2T(n/2) + 1$ for $n > 1$, n a power of 2. The solution and big-O complexity are

(A) $T(n) = n \log n$, $O(n \log n)$

(B) $T(n) = \log n$, $O(\log n)$

(C) $T(n) = n - 1$, $O(n)$

(D) $T(n) = 2^n - 1$, $O(2^n)$

Answer: (C). $T(n) = 2^k T(n/2^k) + 2^k - 1$; at $k = \log n$: $0 + n - 1$.

My answer

Q29. [new]

Solve $T(1) = 0,\ T(n) = 2T(n/2) + 2$ for $n > 1$ (the additions of method g in Q5), n a power of 2.

(A) $T(n) = 2n - 2$, which is $O(n)$

(B) $T(n) = 2 \log n$, which is $O(\log n)$

(C) $T(n) = 2n \log n$, which is $O(n \log n)$

(D) $T(n) = n - 1$, which is $O(n)$

Answer: (A). $T(n) = 2^k T(n/2^k) + 2(2^k - 1)$; at $2^k = n$: $2n - 2$.

My answer

Q30. [new]

What is the closed form of $T(n) = T(n/2) + n$ for $n > 1$, $T(1) = 1$, where n is a power of 2?

(A) $T(n) = n \log n + 1$

(B) $T(n) = 2n - 1$

(C) $T(n) = n + \log n$

(D) $T(n) = n^2$

Answer: (B). $T(n) = T(1) + n + n/2 + \cdots + 2 = 1 + (2n - 2) = 2n - 1 = O(n)$.

My answer

Q31. Recursion compilation (T211)

The running time of a recursive algorithm is $T(1) = a,\ T(n) = 4T(n/2) + n$ for $n > 1$, where a is a constant and $n = 2^k$. Solving by iteration gives

(A) $T(n) = an^2 + n \log n$

(B) $T(n) = an^2 + n^2 - n$

(C) $T(n) = an + n \log n$

(D) $T(n) = an^2 + n^2$

Answer: (B). $T(n) = 4^k T(n/2^k) + n(2^k - 1)$; with $2^k = n$, $4^k = n^2$: $an^2 + n^2 - n = O(n^2)$.

My answer

Q32. Recursion compilation (T172)

Solve the recurrence $T(1) = 1,\ T(n) = 8T(n/2) + n^2$ for $n > 1$, n a power of 2.

(A) $T(n) = n^3 + n^2 \log n$

(B) $T(n) = 2n^3 - n^2$

(C) $T(n) = n^3$

(D) $T(n) = 2n^2 - n$

Answer: (B). $T(n) = 8^k T(n/2^k) + n^2(2^k - 1)$; at $2^k = n$: $n^3 + n^3 - n^2 = 2n^3 - n^2 = O(n^3)$.

My answer

Q33. Recursion compilation (T152)

Solve the merge sort recurrence $T(1) = 0,\ T(n) = 2T(n/2) + (n - 1)$ (n a power of 2).

(A) $T(n) = n \log n + n$, which is $O(n \log n)$

(B) $T(n) = n \log n - n + 1$, which is $O(n \log n)$

(C) $T(n) = n^2 - 1$, which is $O(n^2)$

(D) $T(n) = n - 1$, which is $O(n)$

Answer: (B). $T(n) = 2^k T(n/2^k) + kn - (2^k - 1)$; at $k = \log n$: $n \log n - n + 1$.

My answer

Q34. Midterm 242

Consider the recursive factorial method:

```
long factorial(int n) {
   if (n == 0)
      return 1;
```

```
    else
        return n * factorial(n - 1);
}
```

Its recurrence relation for the number of multiplications is $T(0) = 0,\ T(n) = T(n-1) + 1$ for $n > 0$. The time complexity of this algorithm, $T(n)$, is

(A) $\Theta(1)$

(B) $\Theta(n)$

(C) $\Theta(2^n)$

(D) $\Theta(n^2)$

Answer: (B). $T(n) = T(n-k) + k$; at $k = n$: $T(n) = n$.

My answer

Q35. [new]

Which recurrence describes the number of times the printing statement is executed by the Towers of Hanoi method below?

```
public static void hanoi(int n, char from, char to, char temp){
    if (n == 1)
        System.out.println(from + " --------> " + to);
    else{
        hanoi(n - 1, from, temp, to);
        System.out.println(from + " --------> " + to);
        hanoi(n - 1, temp, to, from);
    }
}
```

(A) $T(1) = 1, \quad T(n) = 2T(n-1) + 1$ for $n > 1$

(B) $T(1) = 1, \quad T(n) = T(n-1) + 2$ for $n > 1$

(C) $T(1) = 0, \quad T(n) = 2T(n-1) + 1$ for $n > 1$

(D) $T(1) = 1, \quad T(n) = 2T(n/2) + 1$ for $n > 1$

Answer: (A). One print in the base case; otherwise two calls on $n-1$ and one print.

My answer

Q36. [new]

How many lines does `hanoi(6, 'A', 'C', 'B')` (the method of the previous question) print?

(A) 31

(B) 36

(C) 63

(D) 64

Answer: (C). $T(n) = 2^n - 1$, so $T(6) = 63$; the method is $O(2^n)$.

My answer

Q37. [new]

The number of element comparisons of the recursive binary search (slides version) is described by $T(0) = 0,\ T(n) = T(\lfloor n/2 \rfloor) + 1$ for $n \geq 1$. What is $T(1000)$?

(A) 9

(B) 10

(C) 11

(D) 500

Answer: (B). $T(n) = \lfloor \log_2 n \rfloor + 1 = 9 + 1 = 10$; binary search is $O(\log n)$.

My answer

Q38. [new]

For the recursive selection sort below, which recurrence gives the number of element comparisons `(x[i] > x[k])`?

```
private static void selectionSort(int[] x, int n) {
   if (n > 1) {
      int maxPos = findMaxPos(x, n - 1);
      swap(x, maxPos, n - 1);
      selectionSort(x, n - 1);
   }
}
private static int findMaxPos(int[] x, int j) {
   int k = j;
   for (int i = 0; i < j; i++)
      if (x[i] > x[k]) k = i;
   return k;
}
```

(A) $T(1) = 0, \quad T(n) = T(n-1) + 1$ for $n > 1$

(B) $T(1) = 0, \quad T(n) = T(n-1) + (n-1)$ for $n > 1$

(C) $T(1) = 1, \quad T(n) = T(n-1) + n$ for $n > 1$

(D) $T(1) = 0, \quad T(n) = 2T(n/2) + (n-1)$ for $n > 1$

Answer: (B). `findMaxPos(x, n - 1)` does $n - 1$ comparisons, then one call on $n - 1$.

My answer

Q39. [new]

Using the recursive selection sort of the previous question, how many element comparisons are carried out to sort an array of 10 elements?

(A) 10

(B) 45

(C) 55

(D) 100

Answer: (B). $T(n) = n(n-1)/2$, so $T(10) = 45$; selection sort is $O(n^2)$.

My answer

Q40. [new]

For the recursive Fibonacci method, the number of additions satisfies $T(n) = T(n-1) + T(n-2) + 1$ with $T(0) = T(1) = 0$. Since $T(n-2) \leq T(n-1)$, we get $T(n) \leq 2T(n-1) + 1$. What upper bound does this give?

(A) $T(n) = O(n)$

(B) $T(n) = O(n^2)$

(C) $T(n) = O(n \log n)$

(D) $T(n) = O(2^n)$

Answer: (D). $2T(n-1) + 1$ unrolls like Towers of Hanoi to about 2^n, so $T(n) = O(2^n)$.

My answer

Part B: Programming

Q41. [new]

Write a recursive method `public static double power(double x, int n)` that returns x^n for $n \geq 0$ using $x^n = x \cdot x^{n-1}$. As comments, give the recurrence relation for the number of multiplications, its closed form and its big-O complexity.

Answer:

```
public static double power(double x, int n) {
    if (n == 0)
        return 1;
    return x * power(x, n - 1);
}
// T(0) = 0
// T(n) = T(n-1) + 1,  n > 0
// T(n) = T(n-k) + k ; stop at k = n  ->  T(n) = n  ->  O(n)
```

My answer

Q42. [new]

Write a recursive method `public static long fastPower(long x, int n)` that computes x^n ($n \geq 0$) using $x^n = (x^{n/2})^2$ when n is even and $x^n = (x^{n/2})^2 \cdot x$ when n is odd (integer division). Make only ONE recursive call per invocation. As comments, give the worst-case recurrence for the number of multiplications and its big-O.

Answer:

```
public static long fastPower(long x, int n) {
    if (n == 0)
        return 1;
    long half = fastPower(x, n / 2);   // one recursive call only
    if (n % 2 == 0)
        return half * half;
    else
        return half * half * x;
}
// Worst case (n odd at every level):
// T(0) = 0
// T(n) = T(n/2) + 2,  n >= 1
// T(n) = T(n/2^k) + 2k ; stop when n/2^k = 1, k = log n
// T(n) = T(1) + 2 log n = 2 log n + 2  ->  O(log n)
```

My answer

Q43. [new]

Write a public method `public static int sum(int[] a)` that calls a private recursive method `sum(int[] a, int low, int high)`. The recursive method splits the range into two halves, sums each half recursively and adds the two results (assume `a` is non-empty). As comments, give the recurrence for the number of additions (assume n is a power of 2), its solution and big-O.

Answer:

```
public static int sum(int[] a) {
    return sum(a, 0, a.length - 1);
}
private static int sum(int[] a, int low, int high) {
    if (low == high)
        return a[low];
    int mid = (low + high) / 2;
    return sum(a, low, mid) + sum(a, mid + 1, high);
}
// T(1) = 0
// T(n) = 2T(n/2) + 1,  n > 1
// T(n) = 2^k T(n/2^k) + (2^k - 1) ; stop at 2^k = n
// T(n) = n*0 + n - 1 = n - 1  ->  O(n)
```

My answer

Q44. [new]

Write a recursive insertion sort in the style of the slides' recursive selection sort: `public static void`

`insertionSort(int[] x)` calls `private static void insertionSort(int[] x, int n)`, which first sorts the first $n-1$ elements recursively and then inserts `x[n-1]` into its correct place. As comments, give the worst-case recurrence for the number of element comparisons, its closed form and big-O.

Answer:

```
public static void insertionSort(int[] x) {
    insertionSort(x, x.length);
}
private static void insertionSort(int[] x, int n) {
    if (n > 1) {
        insertionSort(x, n - 1);
        int last = x[n - 1];
        int j = n - 2;
        while (j >= 0 && x[j] > last) {   // element comparison
            x[j + 1] = x[j];
            j--;
        }
        x[j + 1] = last;
    }
}
// Worst case (array in reverse order): inserting x[n-1] costs n-1 comparisons
// T(1) = 0
// T(n) = T(n-1) + (n-1),  n > 1
// T(n) = T(1) + sum_{j=1}^{n-1} (n-j) = n(n-1)/2  ->  O(n^2)
```

My answer

Q45. [new]

Modify the Towers of Hanoi method so that it prints every move and also **returns** the number of moves: `public static long hanoi(int n, char from, char to, char temp)`. As comments, give the recurrence for the number of moves, its closed form and big-O.

Answer:

```
public static long hanoi(int n, char from, char to, char temp) {
    if (n == 1) {
        System.out.println(from + " --------> " + to);
        return 1;
    }
    long moves = hanoi(n - 1, from, temp, to);
    System.out.println(from + " --------> " + to);
    moves++;
    moves += hanoi(n - 1, temp, to, from);
    return moves;
```

```
}
// T(1) = 1
// T(n) = 2T(n-1) + 1,  n > 1
// T(n) = 2^k T(n-k) + 2^(k-1) + ... + 2 + 1 ; stop at k = n-1
// T(n) = 2^(n-1) + 2^(n-2) + ... + 1 = 2^n - 1  ->  O(2^n)
```

My answer

Q46. [new]

Write `public static boolean isPalindrome(String s)` that calls a private recursive method `isPalindrome(Str s, int low, int high)` which compares the outer characters and recurses on the inner substring. As comments, give the worst-case recurrence for the number of character comparisons (in terms of the length n), its solution and big-O.

Answer:

```
public static boolean isPalindrome(String s) {
    return isPalindrome(s, 0, s.length() - 1);
}
private static boolean isPalindrome(String s, int low, int high) {
    if (low >= high)
        return true;
    if (s.charAt(low) != s.charAt(high))
        return false;
    return isPalindrome(s, low + 1, high - 1);
}
// Worst case (s is a palindrome):
// T(0) = T(1) = 0
// T(n) = T(n-2) + 1,  n >= 2
// T(n) = T(n-2k) + k ; stop when n-2k is 0 or 1, k = floor(n/2)
// T(n) = floor(n/2)  ->  O(n)
```

My answer

Q47. [new]

Write a recursive method `public static int countDigits(int n)` that returns the number of decimal digits of a non-negative integer `n` (`countDigits(0)` is 1). As comments, give the recurrence for the number of divisions, its solution and big-O.

Answer:

```
public static int countDigits(int n) {
    if (n < 10)
        return 1;
    return 1 + countDigits(n / 10);
}
// T(n) = 0                  n < 10
// T(n) = T(n/10) + 1        n >= 10
// T(n) = T(n/10^k) + k ; stop when n/10^k < 10, k = floor(log10 n)
// T(n) = floor(log10 n)  ->  O(log n)
```

My answer

Q48. [new]

Modify the slides' recursive binary search into `public static int lowerBound(int[] array, int target, int low, int high)` that returns the index of the first element of the sorted `array` that is $\geq$ `target` (or `high + 1` of the initial call, i.e. `array.length`, if there is none). As comments, give the recurrence for the number of element comparisons, its solution and big-O.

Answer:

```java
public static int lowerBound(int[] array, int target, int low, int high) {
    if (low > high)
        return low;
    int middle = (low + high) / 2;
    if (array[middle] >= target)
        return lowerBound(array, target, low, middle - 1);
    else
        return lowerBound(array, target, middle + 1, high);
}
// Call: lowerBound(array, target, 0, array.length - 1)
// T(0) = 0
// T(n) = T(n/2) + 1,  n >= 1
// T(n) = T(n/2^k) + k ; stop at n/2^k = 1 -> k = log n
// T(n) = T(1) + log n = 1 + log n  ->  O(log n)
```

My answer

Q49. [new]

Write a complete recursive `public static void mergeSort(int[] a)` with its helper `private static void merge(int[] a, int[] left, int[] right)` that merges the two sorted halves back into `a`. As comments, give the worst-case recurrence for the number of element comparisons (assume n is a power of 2), its solution and big-O.

Answer:

```java
public static void mergeSort(int[] a) {
    if (a.length > 1) {
        int q = a.length / 2;
        int[] left = Arrays.copyOfRange(a, 0, q);
        int[] right = Arrays.copyOfRange(a, q, a.length);
        mergeSort(left);
        mergeSort(right);
        merge(a, left, right);
    }
}
private static void merge(int[] a, int[] left, int[] right) {
    int i = 0, j = 0, k = 0;
    while (i < left.length && j < right.length) {
        if (left[i] <= right[j]) a[k++] = left[i++];   // one comparison
        else a[k++] = right[j++];
    }
    while (i < left.length) a[k++] = left[i++];
    while (j < right.length) a[k++] = right[j++];
}
```

```
// merge does at most n-1 comparisons
// T(1) = 0
// T(n) = 2T(n/2) + (n-1),  n > 1
// T(n) = 2^k T(n/2^k) + kn - (2^k - 1) ; stop at k = log n
// T(n) = n log n - n + 1  ->  O(n log n)
```

My answer

Q50. [new]

Write a recursive method `public static void triangle(int n)` that prints a row of `n` stars, then a row of `n-1` stars, ..., down to a row of one star (use a loop to print each row, and recursion for the remaining rows). As comments, give the recurrence for the number of times `System.out.print("*")` is executed, its closed form and big-O.

Answer:

```
public static void triangle(int n) {
    if (n > 0) {
        for (int i = 0; i < n; i++)
            System.out.print("*");
        System.out.println();
        triangle(n - 1);
    }
}
// T(0) = 0
// T(n) = T(n-1) + n,  n > 0
// T(n) = T(n-k) + (n-k+1) + ... + (n-1) + n ; stop at k = n
// T(n) = 1 + 2 + ... + n = n(n+1)/2  ->  O(n^2)
```

My answer